GUÍA DEL MÉTODO DE CONTINUIDAD v3

Proyecto: Nitrotoon Control de acceso

Cómo seguir el proyecto con IA pública (chat gratuito), paso a paso, sin perder el hilo — ni para la IA ni para ti.

Fecha: 28/10/2026 · Versión de la guía: 1 · La fuente de verdad vive en el repo: github.com/afvelis/nitroacceso

0. Qué es este método y qué archivos lo forman

La idea central cabe en tres líneas: todo el conocimiento del proyecto vive en unos pocos archivos dentro de tu repositorio de GitHub. Cada sesión con la IA arranca pegando dos de esos archivos y termina actualizándolos. Así, ni la IA ni tú pierden el hilo, sin importar qué IA uses (chat gratuito o agente).

Los 5 archivos del método (todos en la RAÍZ del repo, salvo la carpeta tareas):

Archivo	Qué contiene	¿Cuándo lo pegas al chat?
AGENTS.md	Las reglas de trabajo: tu prompt maestro. Incluye el anexo del protocolo de continuidad y el modo chat gratuito.	SIEMPRE, de primero.
ESTADO.md	Dónde estamos: módulos verificados, workstreams activos, siguiente paso, verificación de arranque. Es el único archivo que cambia en cada sesión.	SIEMPRE, de segundo.
ARQUITECTURA.md	Tablas, funciones RPC con sus firmas, manifiesto de archivos, reglas SQL.	Solo si la sesión toca base de datos o RPC.
DECISIONES.md	Los porqués de cada decisión (solo se agrega, nunca se reescribe).	Rara vez: para consultar una decisión vieja.
tareas/ (carpeta)	Backlog ejecutable: una tarea por archivo, con estado (pendiente / en_progreso / hecho).	Cuando trabajas esa tarea concreta.

> *Regla de oro: EL REPO MANDA. Tu carpeta en el PC (D:\NITROTOONProject\moduloAcceso\continuidad\) es una copia de trabajo para pegar en los chats. Si el PC y el repo se diferencian, gana el repo — y en la sección 4 verás cómo dejarlos siempre iguales.*

1. ANTES de abrir el chat (preparación — 5 minutos)

Paso 1.1 — Mira el repo: entra a github.com/afvelis/nitroacceso y revisa dos cosas: que la pestaña Actions muestre los últimos despliegues en verde, y que los archivos del método sigan ahí (AGENTS.md, ESTADO.md, ARQUITECTURA.md, DECISIONES.md, tareas/).

Paso 1.2 — Consigue los archivos para pegar. Dos opciones: (a) tus copias locales en D:\NITROTOONProject\moduloAcceso\continuidad\ — válidas SOLO si las actualizaste al cerrar la sesión anterior; o (b) directo del repo: abre el archivo en GitHub → botón Code → Raw (o el botón de copiar) → selecciona todo y copia.

Paso 1.3 — Cierra pendientes: si la sesión anterior terminó sin decir CERRAR SESIÓN, vuelve a ese chat y ciérrala bien (sección 4) antes de abrir una nueva. Un cierre saltado es la causa número uno de que el método se desincronice.

2. ABRIR el chat y arrancar (modo IA pública)

Paso 2.1 — Abre un chat nuevo (DeepSeek web u otra IA gratuita que uses).

Paso 2.2 — Pega EN ESTE ORDEN, cada uno como mensaje (o juntos si la IA lo permite): (1) el texto de arranque de una línea (texto-antes-inicio-obligatorio.txt); (2) AGENTS.md completo; (3) ESTADO.md completo; (4) SOLO si la sesión toca RPC/tablas: ARQUITECTURA.md; SOLO si van a trabajar una tarea: el archivo de tareas/ correspondiente (ej.: tareas/P0-01-xss-escape.md).

Paso 2.3 — Qué debe hacer la IA: leer todo y ejecutar la Verificación de Arranque (sección 2 del ESTADO.md). Te hará preguntas; tú respondes lo que ves en GitHub: archivos presentes, últimos commits, estados de tareas/ y si el último run del keep-alive está en verde. La IA NO puede entrar a tu GitHub ni a tu Supabase: tú eres sus ojos allí.

Paso 2.4 — Excepción crítica: si la IA dice que algo no cuadra (un archivo que falta, una contradicción entre ESTADO y repo, un paso que no encaja), DETENTE. No sigas, no dejes que adivine ni que reconstruya de memoria. Acláralo primero, aunque sea pegando el archivo que haga falta.

> *Prueba de que el arranque fue bien: la IA te dice en una o dos frases dónde estamos y cuál es el siguiente paso, sin saludar largo y sin resumir todo. Si en cambio empieza a reexplicarte tu propio método, repítelo: pegó incompleto.*

3. DURANTE el trabajo: las reglas que evitan daños

- **Ritmo:** Un paso por mensaje. La IA te propone una acción, tú la ejecutas y reportas lo que viste. No aceptes tres cambios en un solo mensaje.
- **Tamaño:** Cambios de menos de ~30 líneas: la IA entrega bloques Ctrl+F + reemplazar y SIEMPRE te dice cuántas líneas borra y cuántas pega. Cuenta ambas antes de aceptar: si el número no te cuadra, detente y pregunta.
- **Tamaño:** Cambios grandes o riesgosos: archivo completo (abrir → Ctrl+A → pegar el nuevo → guardar). Es tu opción segura cuando el Ctrl+F da miedo.

- **Despliegue:** Si tocó HTML/JS: haces commit en GitHub y ESPERAS el verde de Actions antes de probar (regla M-26). El 90% de los "no se refleja" es un deploy a medio camino.
- **Base de datos:** Si tocó RPC o SQL: TÚ ejecutas el SQL en el SQL Editor de Supabase y pegas el resultado al chat. Para reescribir una función, SIEMPRE primero su definición vigente (Consulta C de ARQUITECTURA.md: pg_get_functiondef) — nunca un SQL de memoria de la IA.
- **Honestidad:** La IA nunca inventa el contenido de un archivo. Si necesita editar un HTML que no pegaste, que te lo pida. Tampoco aceptes URLs inventadas ni datos que no estén en los archivos.
- **Secretos:** JAMÁS pegues claves, tokens, PINs ni contraseñas en ningún chat. La única clave del proyecto (la publishable) ya vive en config.js y es pública por diseño.
- **Cansancio del chat:** Si la sesión se alarga, el chat pesa o notas que la IA empieza a olvidar cosas: pide consolidar y CIERRA SESIÓN. Es mejor cerrar temprano que arriesgar el contexto. El estado vive en los archivos, no en el chat.

4. CERRAR SESIÓN y actualizar GitHub y tu PC (el corazón del método)

Este es el momento que evita que el proyecto se te pierda otra vez como el 22/10. Hazlo completo, siempre, aunque el cansancio diga que mañana.

4.1 — Escribe exactamente en el chat: CERRAR SESIÓN (o "handoff").

4.2 — La IA debe entregarte CUATRO bloques listos para copiar: (a) el ESTADO.md completo regenerado, con versión +1 y fecha nueva; (b) las líneas NUEVAS de DECISIONES.md (si hubo decisiones); (c) los estados actualizados de las tareas/ tocadas, con su evidencia; (d) la lista exacta de archivos que debes actualizar en GitHub. Si falta alguno, pídelo. NO cierres la pestaña del chat sin guardar esos bloques.

4.3 — Actualizar GitHub con el lápiz, archivo por archivo (en este orden):

- ESTADO.md → abrir → lápiz → Ctrl+A → Ctrl+V (pegar el nuevo completo) → Preview para revisar → Commit changes con mensaje tipo "docs: estado v(N+1)".
- DECISIONES.md → abrir → lápiz → ir AL FINAL (Ctrl+Fin) → pegar SOLO LAS LÍNEAS NUEVAS → Commit ("docs: decisiones del día"). Nunca reemplaces DECISIONES completo: es un diario que solo crece.
- tareas/*.md tocadas → cambiar solo la línea "estado:" y pegar la sección "## Notas de implementación" si la IA la entregó.
- El código del día: los HTML/JS que la IA tocó, con el mismo lápiz (archivo completo o los bloques que te dio).

Antes de cada Commit changes: mira el diff (las líneas en verde y rojo que GitHub te muestra). Es tu última defensa contra un pegado en el lugar equivocado. Si ves algo raro, no commitees: pregunta.

4.4 — Esperar el verde en Actions (regla M-26), sobre todo si tocaste HTML/JS.

4.5 — Actualizar tus copias locales (dejar PC = repo): para cada .md que actualizaste, ábrelo en GitHub → botón Code → Raw → Ctrl+A → Ctrl+C → pégalo sobre tu archivo local en D:

\NITROTOONProject\moduloAcceso\continuidad\ y guarda. Alternativa: Code → Download ZIP para bajar el repo completo a tu PC.

4.6 — Si tocó código de la app, ahora sí: prueba con el verde ya puesto y confirma que todo funciona.

5. Peligros comunes y su antídoto

Peligro	Antídoto
Reemplazar DECISIONES.md completo al actualizarlo	DECISIONES solo se AGREGA al final (Ctrl+Fin y pegar lo nuevo). El único que se reemplaza entero es ESTADO.md.
Editar el archivo equivocado (nombres parecidos)	cortesia / cortesias, aliado / aliados, puerta / puerta.html. Lee el nombre DOS veces antes de Ctrl+A.
Commitear sin mirar	El diff de GitHub (verde/rojo) antes de cada Commit changes. Si algo no encaja, no commitees.
Probar con el deploy a medio camino	M-26: primero el verde en Actions, después pruebas.
Borrar una línea de más con Ctrl+F	La IA debe decir cuántas líneas borra y cuántas pega. Cuéntalas en el archivo antes de guardar. Si no coincide: detente.
La IA "recuerda" un archivo que no pegaste	No aceptar. Pedirle que te pida el pegado completo del raw antes de editar.
Pegar secretos en el chat	Jamás: ni tokens, ni PINs, ni contraseñas, ni la clave service_role.
Cerrar la pestaña del chat sin los bloques de cierre	Pedir los 4 bloques ANTES de cerrar. Sin ellos, la sesión no contó.
ESTADO.md dañado o reemplazado por error	Tranquilo: GitHub guarda la historia. Ver la sección 6 (plan de rescate).
Sesión kilométrica que degrada al chat	Cerrar temprano. El método está diseñado para sesiones cortas y frecuentes.

6. Si algo sale mal: plan de rescate

6.1 — Un archivo quedó mal en GitHub (pegaste mal, se dañó, borraste algo): abre el archivo → botón History → elige el commit anterior (cada commit es una foto del archivo) → View file → copia todo → lápiz → pega → Commit. Nada se pierde nunca: siempre puedes volver a una foto anterior.

6.2 — La IA se enredó a mitad de sesión: cierra ese chat sin remordimiento, abre uno nuevo y arranca con el protocolo normal. Mientras hayas commiteado el estado bueno, no perdiste nada.

6.3 — La app no refleja los cambios: mira Actions. Si está corriendo, espera. Si falló, abre el run, mira el error y repórtalo a la IA tal cual (captura o texto).

6.4 — Supabase pausado por inactividad: GitHub → Actions → "Keep Supabase Alive" → Run workflow → espera el verde. El workflow corre solo cada 3 días; esto es el botón manual.

6.5 — Duda general sobre cómo está el proyecto: pega AGENTS.md + ESTADO.md y pide: "ejecuta la verificación de arranque completa y dime qué no cuadra". Esa es la sirena de alarma del método.

7. Chuleta de bolsillo (todo el método en una línea por fase)

Fase	Qué haces
ARRANCA	Chat nuevo → pega AGENTS.md + ESTADO.md (+ ARQUITECTURA.md o la tarea si toca).
VERIFICA	La IA corre la verificación de arranque; tú reportas lo que ves en GitHub.
TRABAJA	Un paso por mensaje; líneas que borra/pega contadas; commit y verde antes de probar.
CIERRA	Escribe CERRAR SESIÓN; guarda los 4 bloques que entrega la IA.
ACTUALIZA	GitHub con el lápiz (ESTADO entero; DECISIONES solo al final; tareas; código) + refresca copias locales con Raw.
RESPIRA	Actions en verde. La próxima sesión arranca sola con los archivos actualizados.

> *Frase para recordar: "el chat se bota, el repo no". Mientras el cierre de sesión quede commiteado, ningún desastre de contexto puede hacerte perder terreno.*

8. Apéndice: trabajar con un agente (cuando uses uno)

Si algún día usas un asistente con acceso al repo (como esta sesión), el método es el mismo con dos diferencias: (1) el agente lee AGENTS.md, ESTADO.md y tareas/ DIRECTO del repo — no necesitas pegar nada; y (2) el agente nunca escribe directo en main: crea una rama agente/fecha-tema, commitea con mensajes [agente] y te deja un Pull Request que TÚ revisas y fusionas. El SQL de Supabase sigue siendo tuyo en todos los casos.

Con IA pública o con agente, el método no cambia de fondo: ARRANCA, VERIFICA, TRABAJA, CIERRA, ACTUALIZA.